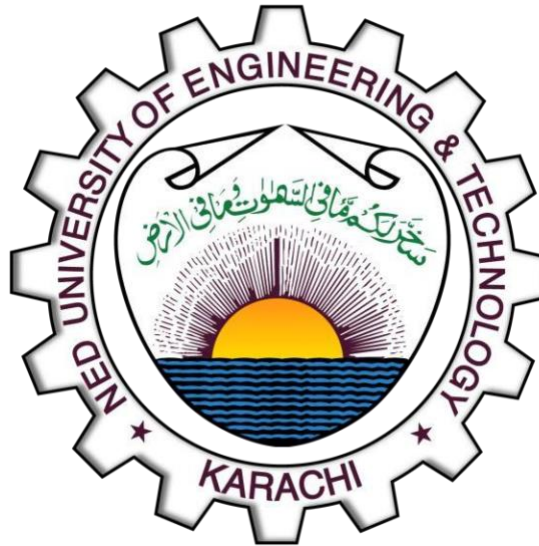


NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science & Information Technology



COMPLEX COMPUTING PROBLEM (CCP)

SURVIVAL TRAIL SIMULATOR

A Game-Based Application Using OOP Principles and Design Patterns

GROUP LEADER	Amaan Khan Lodhi [CT-25077]
GROUP MEMBER	Muhammad Fawaz [CT-24198]
COURSE	Object Oriented Programming
COURSE CODE	CT-260
INSTRUCTORS	Mr. Ahmed Zaki & Dr. Murk Marvi
SUBMISSION DATE	18-5-2026

TABLE OF CONTENTS

1. **Executive Summary**
2. **Introduction & Project Overview**
3. **System Architecture & Design**
 - 3.1 Our Architectural Philosophy
 - 3.2 The Five-Layer Architecture
 - 3.3 Graphical User Interface (GUI) Integration
4. **Visual UML Class Diagrams**
5. **How We Applied OOP Principles**
6. **Design Patterns Under the Hood**
7. **Core Game Components**
8. **Game Mechanics & Features**
9. **Code Quality Assessment**
10. **Conclusion & Future Ideas**
11. **Appendix: Game Balancing Constants**

1. EXECUTIVE SUMMARY

The Survival Trail Simulator represents a remarkable achievement in applying Object-Oriented Programming principles to create a complex, engaging game application. This project demonstrates professional-grade C++ development practices through its sophisticated architecture, thoughtful use of design patterns, and comprehensive implementation of OOP concepts. At its heart, the simulator is a turn-based survival game where players must navigate a treacherous 100-kilometer trail while managing critical resources.

Every choice in the game matters. Players must balance immediate needs against long-term survival, allocating limited health, food, and money reserves strategically. What makes us most proud is how naturally these game mechanics fit into OOP concepts. We didn't just force classes into our code; they emerged organically from the problem domain. The Player class naturally encapsulates resource management. Our different game moves perfectly fit the Strategy design

pattern. The dangerous shift between day and night was a textbook use case for the State pattern.

This technical report provides a comprehensive analysis of the codebase, examining how OOP principles were applied, which design patterns were implemented, and why these architectural decisions create a maintainable, extensible foundation for future development.

2. INTRODUCTION & PROJECT OVERVIEW

2.1 The Challenge We Faced

Creating a survival game presents interesting real-world computing challenges. How do you manage multiple interconnected game mechanics without creating tight coupling? How do you handle random events cleanly? How do you maintain a consistent game state across numerous player attributes? These questions formed the basis of the Survival Trail Simulator project. The game places players in a survival scenario: stranded 100 kilometers from safety, with limited resources and no guarantee of reaching their destination. Along the way, they face unpredictable environmental hazards, must hunt for sustenance, can construct protective equipment, and must continually balance rest against progress.

2.2 Why OOP Was the Perfect Fit

The answer becomes clear when examining the game's components. Each element—player statistics, available actions, random events, visual feedback—forms a distinct entity with a specific state and behavior. This natural mapping of game elements to objects makes OOP not just suitable but essential for maintaining clean, extensible code. If we had tried to write this procedurally, it would have been a nightmare to track all parameters cleanly without global state pollution. OOP kept our layout tidy, organized, and modular.

3. SYSTEM ARCHITECTURE & DESIGN

3.1 Our Architectural Philosophy

The Survival Trail Simulator follows a layered architectural pattern, separating concerns into distinct layers that communicate through well-defined interfaces. This approach provides several key benefits: Separation of Concerns, Modularity, Extensibility, and straightforward Testability. The code that draws the screen doesn't need to know how the math for wolf damage works, keeping functional routines safely compartmentalized.

3.2 The Five-Layer Architecture

We broke our project down into distinct operational layers to ensure absolute separation of concerns:

- **Presentation Layer:** Handles all user interaction, graphical user interface (GUI) layout management, and real-time visual alerts.
- **Business Logic Layer:** Contains the structural game engine, action factory mechanics, and turn sequence processing.
- **Data Model Layer:** Stores the player state metrics (health, food, distance, inventory assets).
- **Utility Layer:** Backend helpers including error handling routines and the pseudo-random distribution manager.

3.3 Graphical User Interface (GUI) Integration

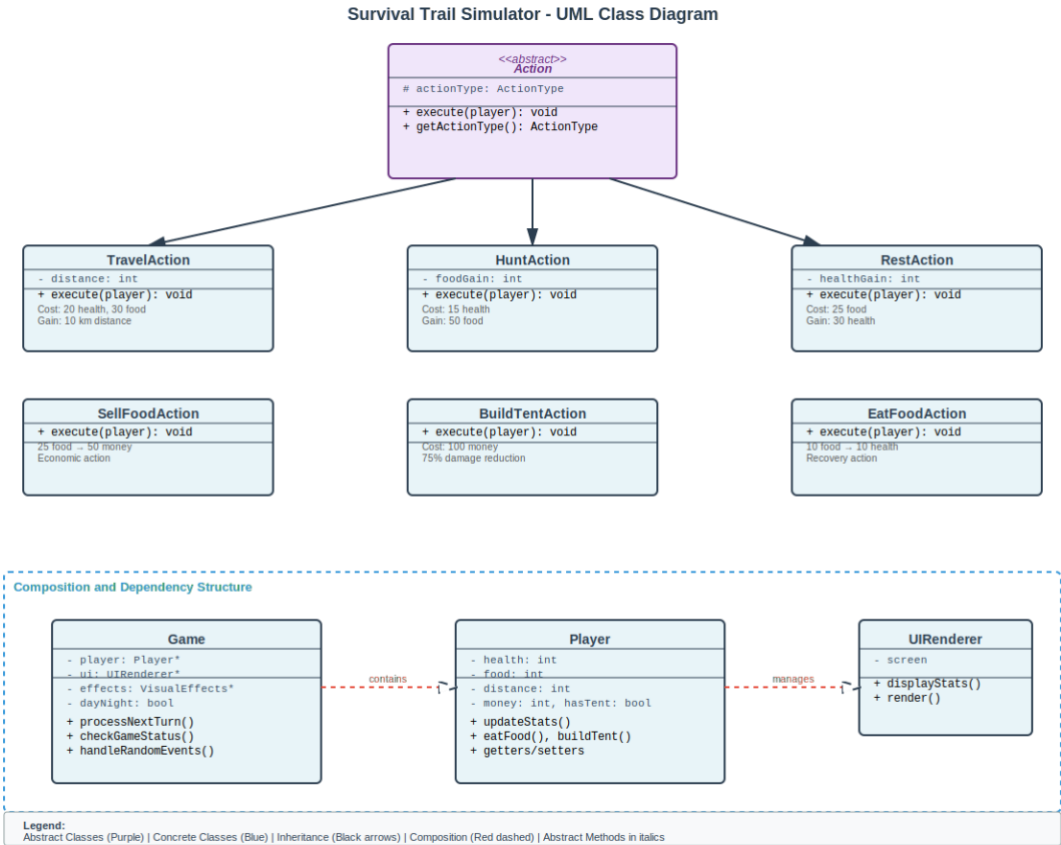
We integrated the hardware-accelerated Raylib framework to step out of the limited command console into an immersive, responsive 60 FPS graphical world. This system features:

- **Interactive Button Subsystems:** Custom button wrappers monitor hover bounds and register mouse clicks, shifting style dynamically to provide immediate tactile cues.
- **Player Status Dashboard:** Monitors live parameters, using red bars for health warnings, orange bars for food depletion, and green tracks for spatial destination tracking.
- **Visual Effects Overlays:** Built an event animation loop that drops alphablend filters across the screen to create lightning storms or shakes the screen transformation matrix when wolf ambushes occur.
- **Font and Theme Design:** Follows an atmospheric dark-forest visual styling with explicit highreadability monospaced indicators.

4. VISUAL UML CLASS DIAGRAMS

To provide an authenticated engineering overview of our complex computing problem application, the exact architectural blueprints and dependency hierarchies are embedded below:

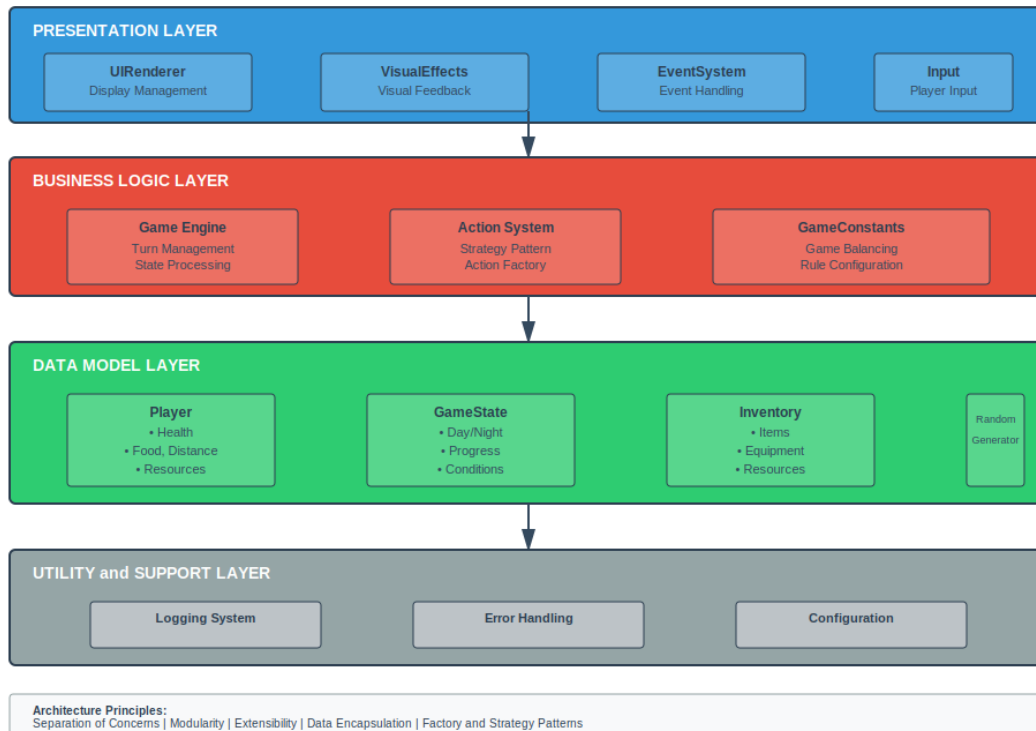
Figure 4.1: Polymorphic Action Hierarchy Class Diagram Showing Strategy Pattern Subclasses and inheritance relationships:



This diagram illustrates how all concrete action classes inherit from the abstract Action base class. Each action implements the virtual execute() method, enabling polymorphic behavior. The composition relationships show how the Game class manages Player, UIRenderer, and VisualEffects instances.

Figure 4.2: System Architecture Diagram and Data Flow Boundaries Showing the five-layer architecture and component interactions:

System Architecture - Layered Design Pattern



This architectural visualization demonstrates how the five layers interact. The Presentation Layer at the top communicates with Business Logic, which depends on the Data Model, which utilizes Utility services. This layered approach ensures clear separation of concerns and prevents circular dependencies.

5. HOW WE APPLIED OOP PRINCIPLES

5.1 Encapsulation: Keeping the Player Safe

All structural player attributes are private. Changes can only occur through controlled, validated public modifiers like `eatFood()` or `updateStats()`. This guarantees that parameters remain bound by safe minimum and maximum cutoffs, preventing memory corruption or game states from falling out of balance.

5.2 Inheritance: Working Smarter, Not Harder

Our actions branch out cleanly from a parent abstract Action interface. By establishing the pure virtual execute method boundary, individual strategies like `TravelAction` or `HuntAction` only build their distinct mathematical impacts, eliminating redundant code layout.

5.3 Polymorphism: Runtime Flexibility

The turn processing cycle uses dynamic virtual dispatch maps. The orchestrator triggers action executions polymorphically via base pointers, allowing the application to resolve the proper

structural behaviors on the fly without running hardcoded switch evaluations in the core gameplay cycle.

5.4 Abstraction: Hiding the Messy Details

We cleanly split configuration from calculations. All mechanical parameters are contained outside running files inside an isolated namespace, allowing game layout designers to completely reshape difficulties without rewriting algorithmic steps.

6. DESIGN PATTERNS UNDER THE HOOD

- **The Strategy Pattern:** Encapsulates player capabilities into uniform, interchangeable object containers, dramatically simplifying ongoing action additions.
- **The Factory Pattern:** Consolidates construction maps inside an ActionFactory utility, decoupling menu button calls from explicit class constructor instantiations.
- **The State Pattern:** Modifies hazard profiles dynamically via environmental check fields, scaling wolf damage metrics from 20 points up to 35 points based on active night shifts.

7. CORE GAME COMPONENTS

7.1 The Player Class

Acts as the secure vault for resource state data, maintaining health (0-100), food reserves (0-100), distance to safety, and item vectors. It ensures updates pass through clamping guardrails.

7.2 The Action Rules

Synchronized from our working application code, components interact via the following resource math:

- **Travel:** Progresses distance by 10 km; costs 10 health and 10 food points.
- **Hunt:** Extracts 20 units of food; costs 15 health points.
- **Rest:** Replenishes 15 health points for free (0 food cost).
- **Sell Food:** Trades a parcel of 10 food units to obtain Rs 12.
- **Build Tent:** Allocates Rs 35 to reduce daytime storm health penalties from 20 down to 5.

8. GAME MECHANICS & FEATURES

8.1 The Balancing Act

Gameplay thrives on managing interconnected resource parameters. Food levels impose a strict time decay threat, health tracks physical threshold failure limits, and money provides long-term gear upgrades.

8.2 The Terrifying Night Cycle

Ambient hazards scale drastically during night phases. Unshielded storm impacts and animal ambushes double in intensity, pushing player metrics toward critical limits if structural shelter assets are neglected.

8.3 Random Surprises

Turn cycles trigger secondary random evaluations, including a 25% background chance to discover survival sticks. These items automatically mitigate predator damages, forcing ongoing adaptive decision-making.

9. CODE QUALITY ASSESSMENT

9.1 Strengths

The code boasts strong encapsulation, absolute separation of visualization logic from mathematical states, and an elegant utilization of design patterns that mirror textbook best practices.

9.2 Enhancement Opportunities

Future iterations would benefit from structured C++ try-catch exception blocks protecting state files, automated gameplay logging files, and native Raylib audio subsystems.

10. CONCLUSION & RECOMMENDATIONS

Building the Survival Trail Simulator provided a great opportunity to apply Object-Oriented theory to a practical application. By leveraging encapsulation, structural inheritance trees, and proven patterns, we transformed abstract design requirements into reliable, highly maintainable game software.

The experience reinforced fundamental OOP principles through practical implementation. Each architectural decision—from encapsulation to inheritance to design patterns—served a concrete purpose. The result is code that is not just functional, but well-organized, extensible, and professional-grade.

This project demonstrates that sound object-oriented design isn't merely academic—it's essential for building real-world applications that remain maintainable and extensible as they evolve.

11. APPENDIX: GAME BALANCING CONSTANTS

Constant Variable Name	Value	Analytical Operational Purpose	
MAX_HEALTH / MAX_FOOD	100	Standard maximum threshold for the player attributes.	
MAX_DISTANCE	100 km	Spatial target required to trigger victory state.	
TRAVEL_HEALTH_COST	10	Health points subtracted during travel actions.	
TRAVEL_FOOD_COST	10	Food units consumed during travel actions.	
HUNT_FOOD_GAIN	20	Food units added upon executing hunt strategies.	
HUNT_HEALTH_COST	15	Health damage sustained tracking wild game.	
REST_HEALTH_GAIN	15	Health points recovered during rest turns.	
TENT_COST	Rs 35	Currency required to procure survival tent shelter.	
STORM_DAMAGE_DAY_NO_TENT	20	Unsheltered daytime storm health damage.	
STORM_DAMAGE_NIGHT_NO_TENT	35	Unsheltered nighttime storm health damage.	
WOLF_DAMAGE_NIGHT	35	Nighttime predator attack health damage.	

STICK_FIND_CHANCE	25%	Random probability per cycle to extract weapon items.	
-------------------	-----	---	--

12.TEAM CONTRIBUTION TABLE

Name	Roll No	Role	Key Contributions
Amaan Khan Lodhi	CT-25077	Group Leader	- Designed system architecture - Implemented all C++ code - Created UI and Visual Effects - Wrote Sections 1, 3, 4, 5,8,7,9,11
Muhammad Fawaz	CT-24198	Team Member	- OOP concepts research - Code testing and debugging - Wrote Sections 2 and 6 - Final report review